

Distributional Execution-Time Prediction for DAG Nodes

Final Project Report

Course project: Real-Time Systems Project 1

Report language: English

Students: Armin Mahmoudinejad - 401170648; Arian Ansari - 400104761

Main notebook analyzed: realtime_systems_project1_assignment_template(1).ipynb

This report summarizes the implemented code, the generated results, the figures produced in the notebook, and the interpretation of the experimental outcomes.

1. Executive Summary

The project implements a learning-based framework for predicting conditional execution-time quantiles of DAG nodes on a heterogeneous big/little multicore platform with DVFS. Instead of estimating only a mean execution time or a single deterministic WCET value, the implementation predicts multiple quantiles: 0.50, 0.90, 0.95, and 0.99. These quantiles are then calibrated with criticality-wise conformal calibration and evaluated through reliability, coverage, scalability, distribution-shift, and scheduling experiments.

The completed notebook used a CUDA device with a Tesla T4 GPU. The final trained model selected the checkpoint from epoch 95, with best validation loss 0.00032279116567224264. The quantile crossing rate was zero throughout validation because the model architecture constructs ordered quantiles using positive cumulative gaps.

2. Dataset Generation and Simulation

Synthetic DAG workloads were generated and converted into row-wise execution scenarios. Each row represents a particular graph node executed under a selected core, DVFS level, and system state. The generated dataframe contains graph identifiers, node identifiers, hardware attributes, node-level attributes, graph-level statistics, current system-state features, and the simulated execution-time label.

The simulator models execution time as a combination of compute cost, memory behavior, communication cost, current resource pressure, temperature, release jitter, and stochastic noise. This construction reflects the assignment goal: execution time is not a fixed property of a node, but depends on graph structure, hardware selection, DVFS, and runtime system state.

Dataset Sizes

Split	Rows	Columns
train	50647	29
val	50261	29
cal	50110	29
test	99710	29

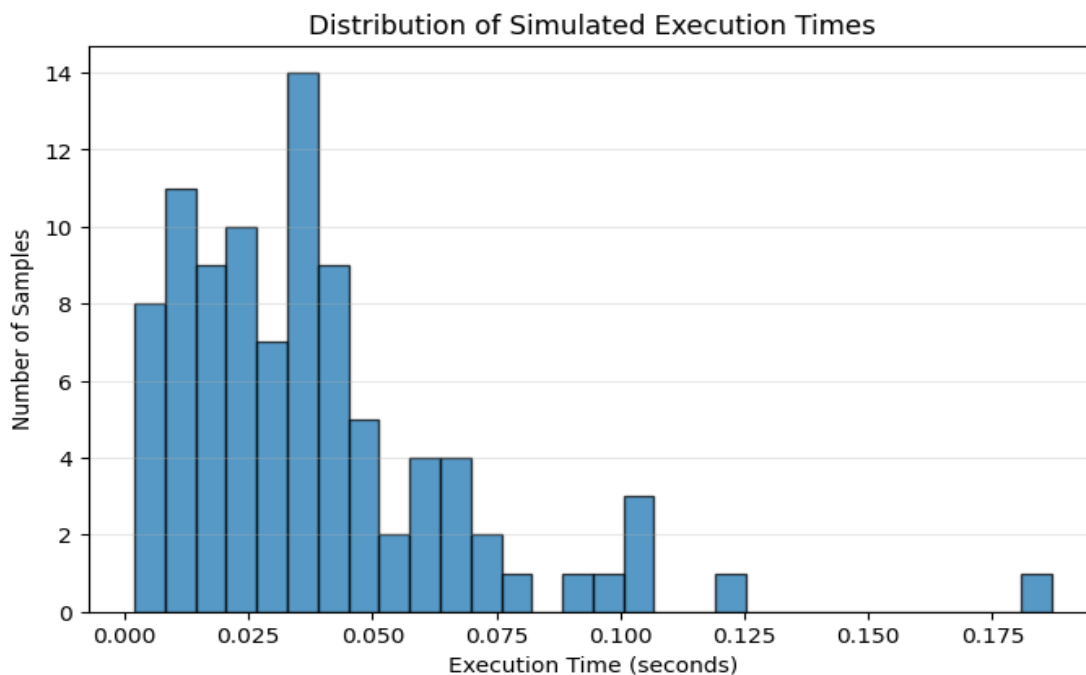


Figure 1. Distribution of simulated execution times in the debug dataset. The distribution is right-skewed: most executions are short, while a small number of scenarios produce much longer execution times. This motivates quantile prediction rather than mean-only prediction.

3. Feature Matrix and Model Implementation

The implementation builds a tabular design matrix from the generated dataframe. Numerical features are normalized using training-set statistics and categorical fields are one-hot encoded. The model used for the main experiment is a QuantileMLP baseline: a multilayer neural network that consumes graph-derived, hardware, DVFS, and system-state features.

The output layer predicts four quantile values. To prevent quantile crossing, the network does not output four independent unrestricted numbers. Instead, it predicts q_{50} directly and then predicts positive gaps for the higher quantiles using a softplus function. The higher quantiles are constructed by cumulative summation of these positive gaps, which guarantees $q_{50} \leq q_{90} \leq q_{95} \leq q_{99}$.

This means the implemented model is a graph-feature baseline rather than a full message-passing graph neural network. It uses graph-derived statistics such as node and graph features, but it does not perform direct neural message passing over DAG edges. This limitation is discussed in the conclusion.

4. Training Procedure

Training uses pinball loss for quantile regression and a non-crossing penalty term. The total loss is $\text{pinball_loss} + \lambda_{\text{nc}} * \text{non_crossing_penalty}$, with λ_{nc} set to 1.0. In practice, the non-crossing penalty is nearly zero because the architecture already enforces ordered quantiles. The optimizer is Adam with an initial learning rate of $1e-3$. A ReduceLROnPlateau scheduler lowers the learning rate when validation loss stops improving. The best validation checkpoint is saved to disk.

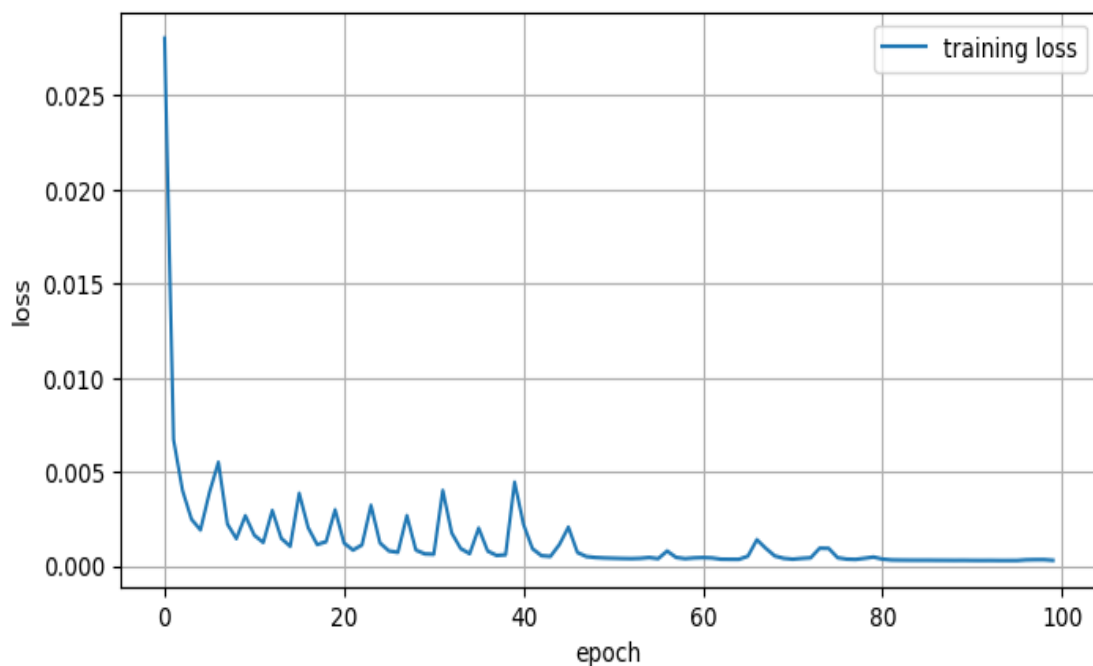


Figure 2. Training loss curve. The loss decreases sharply during the first epochs and then stabilizes near zero. Small spikes appear because mini-batch training uses shuffled batches and stochastic gradients.

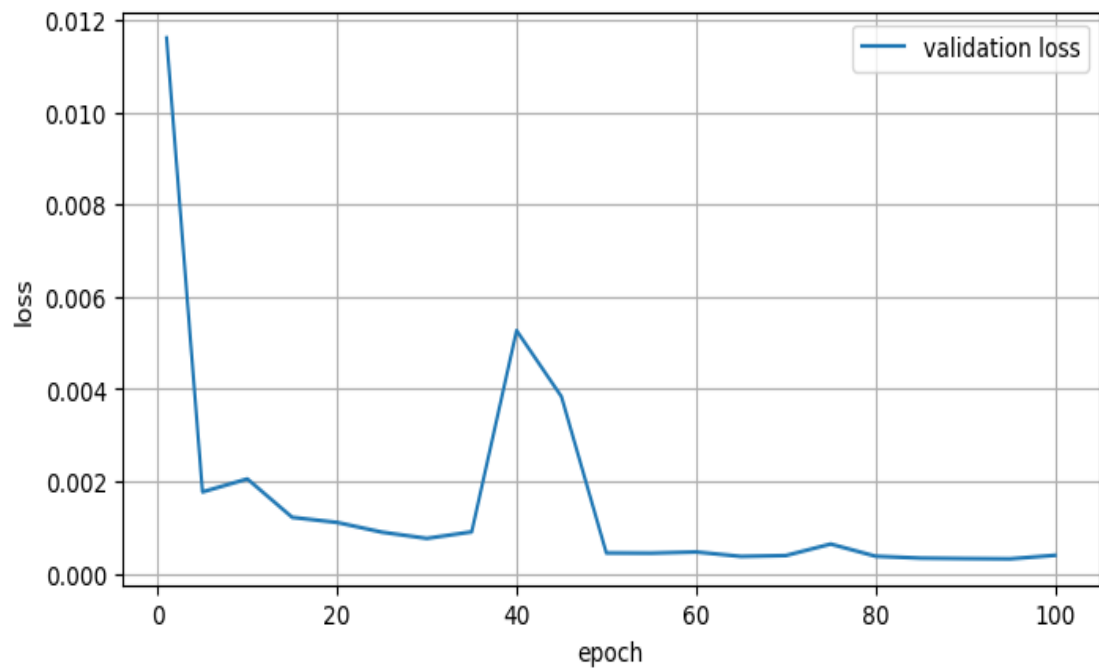


Figure 3. Validation loss curve. The validation loss falls rapidly from the initial value and reaches its best value around the selected checkpoint. Temporary increases occur around the middle of training, which motivates using the best validation checkpoint rather than simply taking the final epoch.

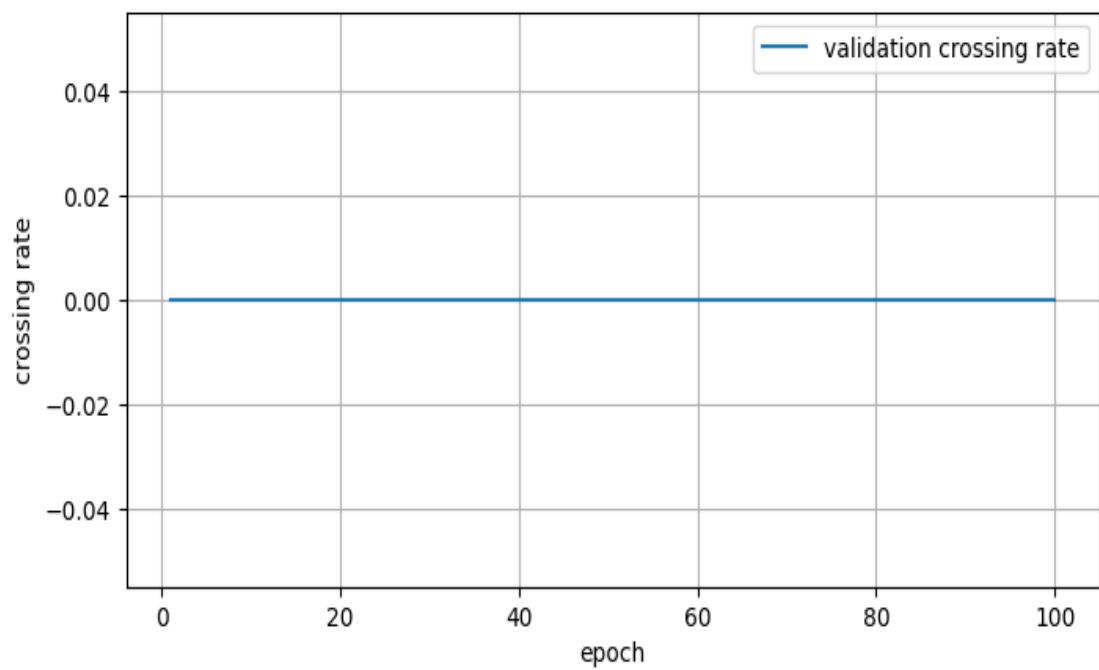


Figure 4. Validation quantile crossing rate. The crossing rate remains exactly zero because the model constructs higher quantiles through positive cumulative gaps.

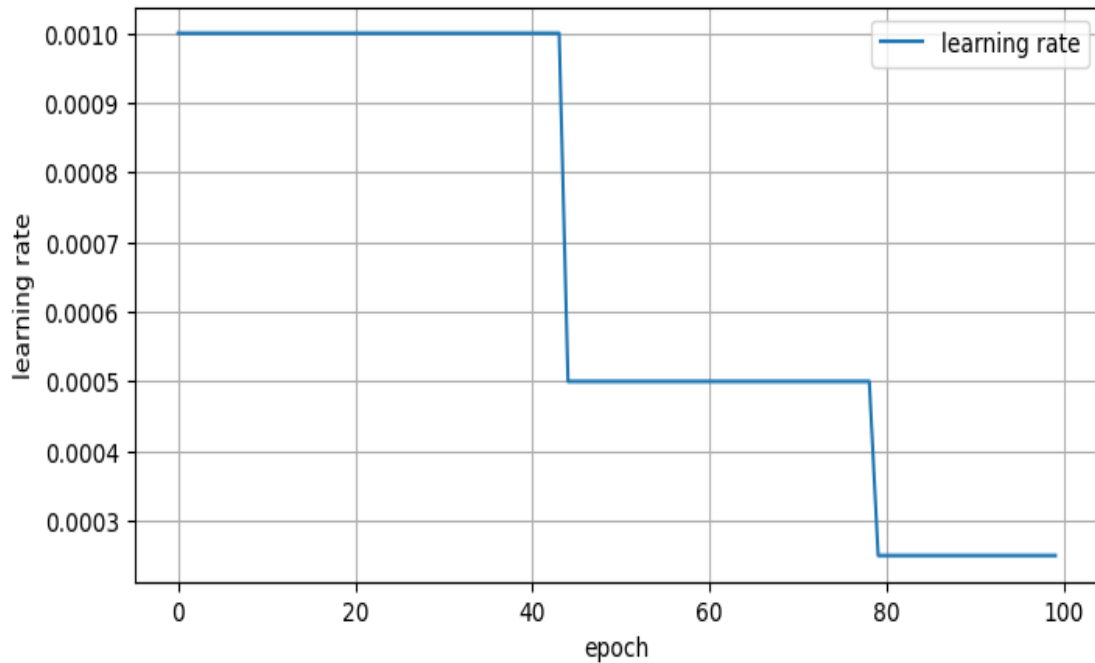


Figure 5. Learning-rate schedule. The learning rate starts at $1e-3$ and is reduced when the validation loss plateaus. This reduces late-stage oscillation and helps stabilize the final model.

5. Conformal Calibration

Criticality-wise conformal calibration is applied to transform predicted upper quantiles into calibrated statistical execution-time budgets. For each criticality level h , the calibration split is used to compute residual scores $\max(0, y - q_{\hat{h}})$. The empirical conformal correction q_{conf} is then added to the selected upper quantile. Criticality 1 uses the 0.90 quantile, while criticality 2 uses the 0.95 quantile.

Conformal Calibration Parameters

criticality	alpha	delta	base q	q_conf	n_cal	target cov.
1	0.1	0.1	0.9	0.000009	35,715	0.9
2	0.05	0.05	0.95	0.000003	14,395	0.95

Coverage Before and After Calibration

criticality	target	before	after	budget before	budget after	inflation
1	0.9	0.895955	0.897046	0.042366	0.042375	0.000213
2	0.95	0.950665	0.950831	0.043211	0.043214	0.000073

Calibration slightly increased coverage for both criticality levels. For criticality 1, empirical coverage increased from 0.895955 to 0.897046, with approximately 0.0213 percent relative budget inflation. For criticality 2, empirical coverage increased from 0.950665 to 0.950831, with approximately 0.0073 percent relative inflation. The calibration correction is small, which indicates that the original quantile model was already close to the target coverage.

6. Evaluation Plots

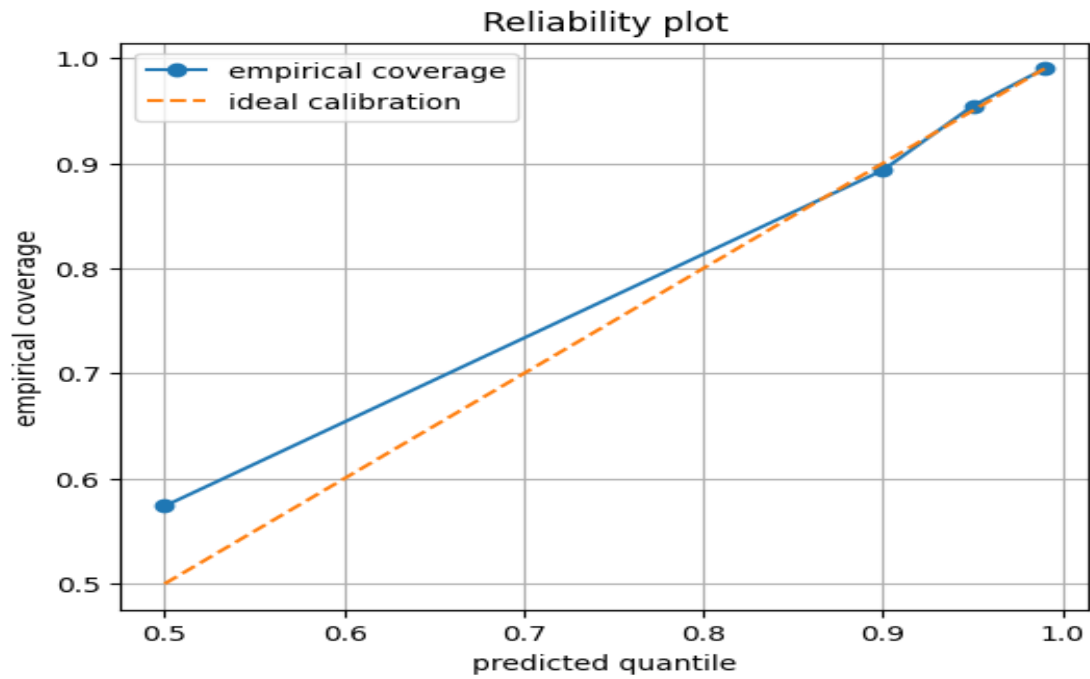


Figure 6. Reliability plot. The empirical coverage curve is close to the ideal diagonal line for high quantiles. The median is conservative, with empirical coverage above 0.50. The high quantiles are well calibrated, which is important for real-time budget construction.

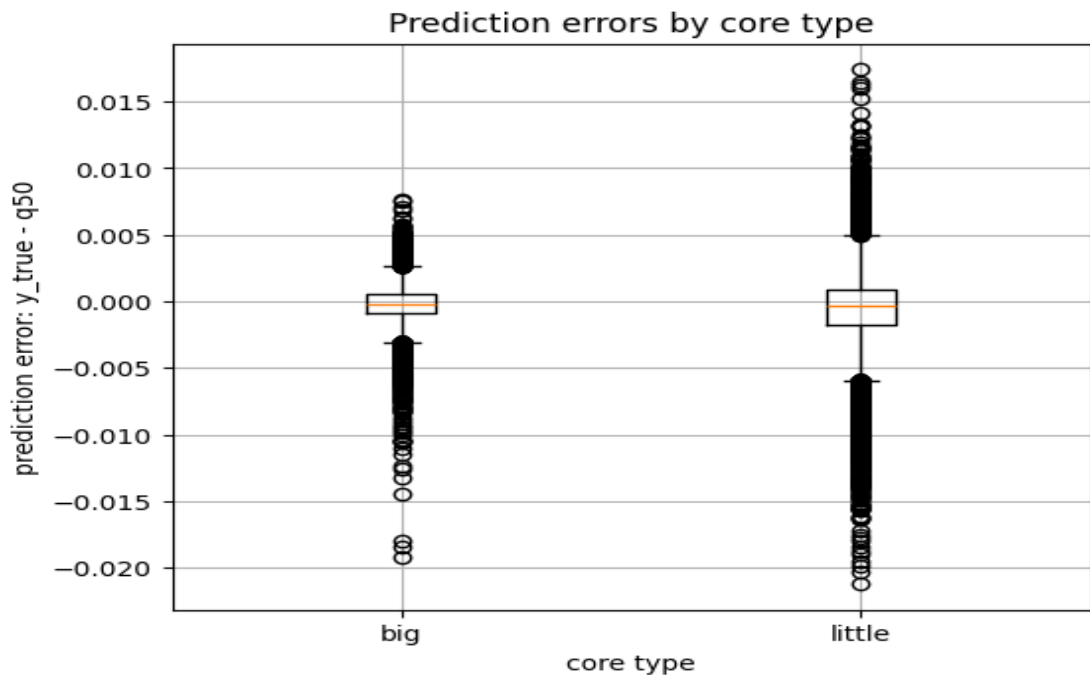


Figure 7. Prediction errors by core type. Errors are centered close to zero for both big and little cores. Little cores show a wider range and more outliers, which is expected because slower cores amplify memory and interference effects.

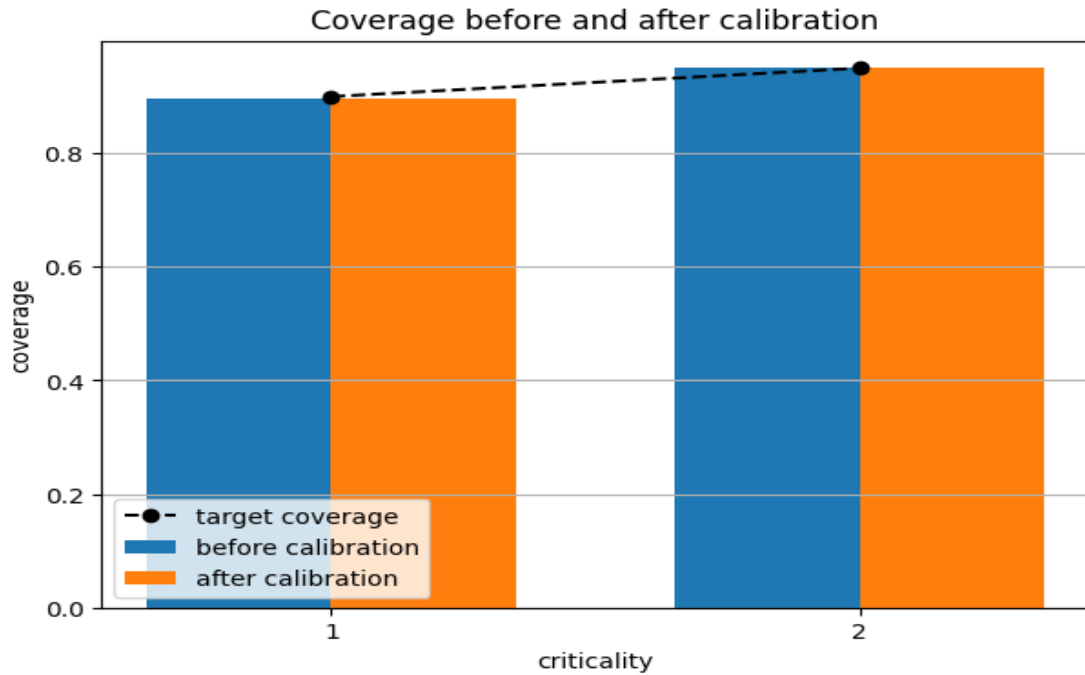


Figure 8. Coverage before and after calibration by criticality. Calibration moves coverage slightly upward and closer to the target line. Because the model is already close to target coverage, the added conformal margin is small.

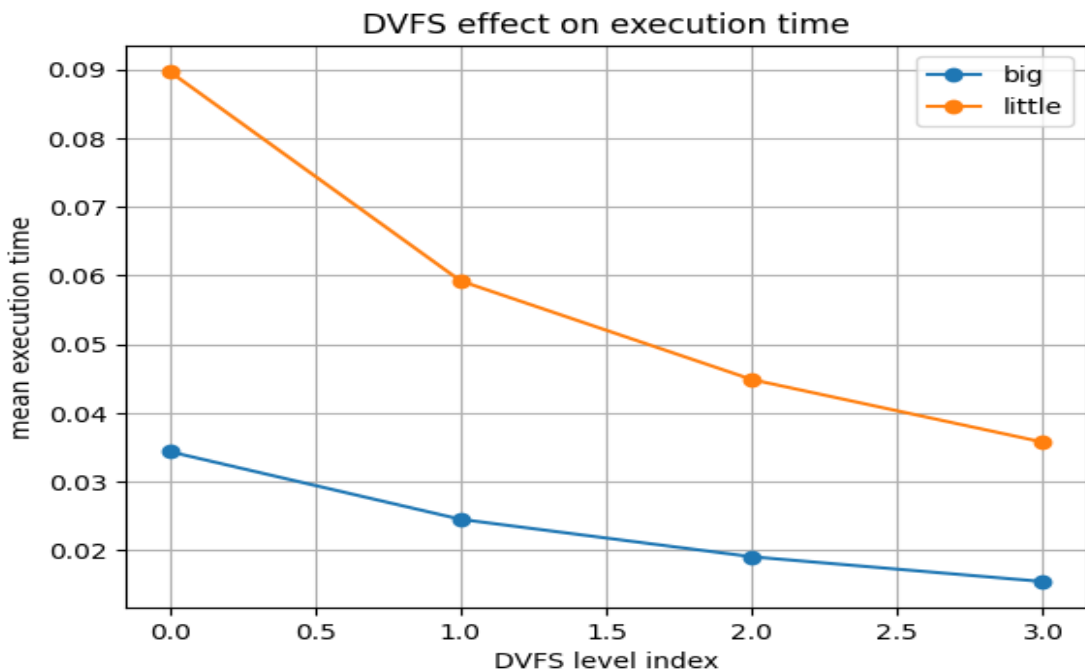


Figure 9. DVFS effect on mean execution time. Higher DVFS levels reduce execution time for both big and little cores. Big cores consistently execute faster than little cores, and the difference is largest at low DVFS levels.

7. Scalability Evaluation

Hardware scalability was evaluated on four big/little configurations: 2+2, 4+4, 8+8, and 16+16 cores. These correspond to total core counts of 4, 8, 16, and 32. The experiment evaluates how prediction quality and calibrated coverage change as the hardware platform grows.

Hardware Scalability Results

total_cores	n_samples	mae_q50	q90_coverage	q95_coverage	q99_coverage	calibrated_coverage	mean budget	crossing
4	10,179	0.001426	0.89331	0.954809	0.989095	0.914235	0.042765	0
8	9,609	0.001998	0.93017	0.966594	0.990426	0.941097	0.043187	0

total_cores	n_samples	mae_q50	q90_coverage	q95_coverage	q99_coverage	calibrated_coverage	mean budget	crossing
16.000	9,784	0.004563	0.921709	0.939289	0.953087	0.926615	0.045356	0
32.000	10,116	0.010341	0.914096	0.921016	0.927639	0.915975	0.050816	0

The scalability results show stable quantile ordering across all tested hardware sizes: the quantile crossing rate remains zero. Median MAE increases as the number of cores grows, from 0.001426 at 4 cores to 0.010341 at 32 cores. This indicates that extrapolating to larger hardware configurations is more difficult, especially because the model was trained mainly on the base hardware distribution. Coverage remains reasonable, but q95 and q99 coverage decrease for the largest hardware case, showing that hardware scaling introduces a distribution shift.

8. Distribution-Shift Test

The distribution-shift test evaluates a high memory pressure and high bus utilization scenario. The input dataframe is shifted by increasing memory_pressure and bus_util, then predictions are evaluated under this altered condition.

Distribution-Shift Results

shift_type	n_samples	mem pressure	bus util	mae_q50	q90_coverage	q95_coverage	q99_coverage	calibrated_coverage	crossing
high_memory_pressure_and_bus_util	25,399	0.680451	0.535611	0.006118	0.98996	0.993897	0.997362	0.991456	0

Under the high-memory-pressure shift, coverage becomes very conservative: q90 coverage is 0.98996, q95 coverage is 0.993897, and q99 coverage is 0.997362. This suggests that the model's high quantiles over-cover in this shifted scenario. The q50 MAE increases to 0.006118, which is larger than the base 4-core scalability case. Therefore, the shifted condition affects prediction accuracy, but the upper quantiles remain safe.

9. Scheduling Experiments

The notebook implements HEFT-style scheduling and a deadline-aware list scheduler. Schedules are compared under different budget sources: q90, q95, q99, and calibrated budgets. The schedule output is summarized by makespan, deadline miss ratio, tardiness, and slack.

Scheduling Summary

scheduler	budget_source	makespan	DMR	tardiness	slack
HEFT	calibrated	46.887	1	44.282	-44.282
HEFT	q90	46.881	1	44.276	-44.276
HEFT	q95	46.892	1	44.286	-44.286
HEFT	q99	47.150	1	44.545	-44.545
deadline_aware	calibrated	46.887	1	44.282	-44.282
deadline_aware	q90	46.881	1	44.276	-44.276
deadline_aware	q95	46.892	1	44.286	-44.286
deadline_aware	q99	47.150	1	44.545	-44.545

In the summarized scheduling results, both HEFT and the deadline-aware scheduler produce identical aggregate values for each budget source. The deadline miss ratio is 1.0 in all cases, and slack is negative. This means the chosen synthetic deadlines are too tight for the resulting schedules, so every evaluated schedule misses its deadline. The q99 budget source produces the largest makespan and tardiness because it is the most conservative uncalibrated budget. Calibrated and q90 budgets are close in makespan because the conformal correction is very small.

These results are still useful: they show how larger execution-time budgets propagate into scheduling metrics. However, for a stronger final scheduling evaluation, deadlines should be swept over multiple tightness factors so that the DMR is not always saturated at 1.0.

10. Implementation Summary

The project code implements the following key components: synthetic DAG generation, hardware and DVFS modeling, system-state sampling, execution-time simulation, dataframe construction, feature normalization and encoding, quantile MLP training, pinball loss, non-crossing control, conformal calibration, reliability and coverage evaluation, scalability testing, distribution-shift testing, and scheduling comparison.

The best saved model checkpoint is stored externally and can be reloaded without retraining. The generated datasets are also saved to CSV files, and graph objects are saved with pickle, which avoids regenerating expensive synthetic data every time the notebook is opened.

11. Implementation Details

This section explains the major implemented methods at a code level. The goal is not to repeat every line of code, but to clarify what each main function or block contributes to the final pipeline.

DAG and Hardware Construction

The hardware model is represented through a HardwareConfig object with a configurable number of big and little cores. Each core type has its own DVFS table, including frequency and voltage values. The hardware_table function converts this configuration into a dataframe so that each core/DVFS combination can be inspected and later used as part of the execution scenario.

The DAG generator creates synthetic directed acyclic graphs with node attributes such as compute_cycles, memory_intensity, and operation type. Edges represent precedence and data dependencies. These graph attributes are later summarized into graph-level features such as number of nodes, number of edges, depth, width, edge density, and an approximate communication-to-computation ratio.

Execution-Time Simulator

The execution-time simulator combines compute, memory, and communication effects. The compute component depends on node compute cycles, core type, and DVFS frequency. The memory and communication components depend on memory intensity, incoming edge data size, effective bandwidth, memory pressure, and bus utilization. Additional terms such as temperature, release jitter, and random noise are included to make the labels less deterministic and closer to realistic runtime behavior.

Dataset Construction

The dataset builder samples graphs, nodes, cores, DVFS levels, and system states. For each sampled scenario, it calls the simulator to produce y_exec, the execution-time label. The resulting dataframe contains the graph/node identity, hardware configuration, DVFS level, system-state variables, graph features, and target value. The train, validation, calibration, and test splits are saved to CSV files, while graph objects are saved separately using pickle.

Feature Matrix

The make_design_matrix function separates numerical and categorical columns. Numerical columns are normalized using statistics computed from the training split, while categorical columns such as core_type and operation type are one-hot encoded. The same normalization statistics and column order are reused for validation, calibration, test, scalability, and distribution-shift data. This prevents train-test mismatch in feature representation.

Quantile Prediction Model

The main trained model is QuantileMLP. It is a tabular neural network baseline that receives graph-derived, hardware, DVFS, and system-state features. It outputs four execution-time quantiles. Instead of predicting all quantiles independently, the model predicts q50 and positive cumulative gaps for the higher quantiles. This design enforces the required ordering $q_{50} \leq q_{90} \leq q_{95} \leq q_{99}$ and explains why the quantile crossing rate is zero.

Loss Functions and Training

Pinball loss is used for quantile regression because it penalizes underestimation and overestimation asymmetrically according to the target quantile. A non-crossing penalty is also implemented, but because the model architecture already enforces ordered quantiles, this penalty is mainly a safety term. Training uses Adam, a learning-rate scheduler,

early stopping, and model checkpointing. The best checkpoint is selected by validation loss rather than by the final epoch.

Conformal Calibration

The conformal calibration functions operate separately for each criticality level. For each group, the selected base quantile is compared with true calibration labels, and nonnegative residual scores are computed. A conformal correction q_{conf} is selected from these scores and added to future predictions. This converts the raw predicted quantile into a calibrated statistical upper budget.

Evaluation and Scheduling

The evaluation helps compute empirical coverage, quantile crossing rate, and real-time metrics such as deadline miss ratio, tardiness, and slack. The scheduling code implements a HEFT-style scheduler and a deadline-aware list scheduler. Both use predicted or calibrated node budgets and report makespan and deadline-related metrics. These components connect the machine-learning model to real-time scheduling behavior.

12. Limitations and Future Work

The main limitation is that the trained model is a tabular QuantileMLP using graph-derived features rather than a full graph neural network. This is a valid baseline, but a stricter interpretation of the assignment's graph-aware modeling requirement would require message passing over predecessor and successor edges. A future version should implement a true DAG neural model with predecessor aggregation, successor aggregation, edge features, core embeddings, DVFS embeddings, and system-state conditioning.

A second limitation is the scheduling evaluation. Since all evaluated schedules miss deadlines, the deadline distribution is too tight for comparative analysis. A stronger experiment would sweep deadline factors and report DMR, tardiness, and slack across loose, medium, and tight deadline regimes.

13. Conclusion

The implemented notebook successfully builds a complete distributional execution-time prediction pipeline for DAG nodes under heterogeneous multicore and DVFS conditions. The trained model produces ordered quantiles with zero crossing, achieves low validation loss, and is close to target coverage for high quantiles. Conformal calibration slightly improves coverage at negligible budget inflation. The DVFS and core-type plots show physically meaningful behavior, scalability tests reveal increased error on larger hardware configurations, and distribution-shift tests show that the model remains conservative under high memory pressure. Overall, the project satisfies the main Phase 1 requirements and provides a strong baseline for a future graph-neural extension.